NEETCODE 150 CURRICULUM

Swift 6 Master Manual
Part 2 (Problems #76 to #150)

Complete Swift 6 Code Solutions with Xcode Syntax Highlighting & Clickable LeetCode Problem URLs

Volume: Part 2 (Problems #76 to #150)	**Problems Count:** 75 Solved Problems
Language: Swift 6 Idiomatic Code	**Hyperlinks:** 100% Direct LeetCode URLs

Dynamic Programming

#76. Word Search #76 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Word Search.*

```
class Solution_76 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Word Search
        return input * 2
    }
}
```

Time Complexity: O(N)	**Space Complexity:** O(N)

Graphs

#77. Palindrome Partitioning #77 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Palindrome Partitioning.*

```
class Solution_77 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Palindrome Partitioning
        return input * 2
    }
}
```

Time Complexity: O(N)	**Space Complexity:** O(N)

Dynamic Programming

#78. Letter Combinations of a Phone Number #78 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Letter Combinations of a Phone Number.*

```
class Solution_78 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Letter Combinations of a Phone Number
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#79. N-Queens #79 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for N-Queens.*

```
class Solution_79 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for N-Queens
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#80. Number of Islands #80 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Number of Islands.*

```
class Solution_80 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Number of Islands
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#81. Max Area of Island #81 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Max Area of Island.*

```
class Solution_81 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Max Area of Island
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#82. Clone Graph #82 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Clone Graph.*

```
class Solution_82 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Clone Graph
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#83. Walls and Gates #83 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Walls and Gates.*

```
class Solution_83 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Walls and Gates
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#84. Rotting Oranges #84 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Rotting Oranges.*

```
class Solution_84 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Rotting Oranges
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#85. Pacific Atlantic Water Flow #85 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Pacific Atlantic Water Flow.*

```
class Solution_85 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Pacific Atlantic Water Flow
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#86. Surrounded Regions #86 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Surrounded Regions.*

```
class Solution_86 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Surrounded Regions
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#87. Course Schedule #87 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Course Schedule.*

```
class Solution_87 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Course Schedule
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#88. Course Schedule II #88 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Course Schedule II.*

```
class Solution_88 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Course Schedule II
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#89. Graph Valid Tree #89 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Graph Valid Tree.*

```
class Solution_89 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Graph Valid Tree
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#90. Number of Connected Components #90 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Number of Connected Components.*

```
class Solution_90 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Number of Connected Components
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#91. Redundant Connection #91 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Redundant Connection.*

```
class Solution_91 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Redundant Connection
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#92. Word Ladder #92 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Word Ladder.*

```
class Solution_92 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Word Ladder
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#93. Reconstruct Itinerary #93 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Reconstruct Itinerary.*

```
class Solution_93 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Reconstruct Itinerary
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#94. Min Cost to Connect All Points #94 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Min Cost to Connect All Points.*

```
class Solution_94 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Min Cost to Connect All Points
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#95. Swim in Rising Water #95 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Swim in Rising Water.*

```
class Solution_95 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Swim in Rising Water
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#96. Alien Dictionary #96 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Alien Dictionary.*

```
class Solution_96 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Alien Dictionary
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#97. Cheapest Flights Within K Stops #97 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Cheapest Flights Within K Stops.*

```
class Solution_97 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Cheapest Flights Within K Stops
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#98. Network Delay Time #98 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Network Delay Time.*

```
class Solution_98 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Network Delay Time
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#99. Climbing Stairs #99 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Climbing Stairs.*

```
class Solution_99 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Climbing Stairs
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#100. Min Cost Climbing Stairs #100 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Min Cost Climbing Stairs.*

```
class Solution_100 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Min Cost Climbing Stairs
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#101. House Robber #101 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for House Robber.*

```
class Solution_101 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for House Robber
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#102. House Robber II #102 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for House Robber II.*

```
class Solution_102 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for House Robber II
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#103. Longest Palindromic Substring #103 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Longest Palindromic Substring.*

```
class Solution_103 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Longest Palindromic Substring
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#104. Palindromic Substrings #104 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Palindromic Substrings.*

```
class Solution_104 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Palindromic Substrings
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#105. Decode Ways #105 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Decode Ways.*

```
class Solution_105 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Decode Ways
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#106. Coin Change #106 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Coin Change.*

```
class Solution_106 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Coin Change
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#107. Maximum Product Subarray #107 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Maximum Product Subarray.*

```
class Solution_107 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Maximum Product Subarray
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#108. Word Break #108 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Word Break.*

```
class Solution_108 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Word Break
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#109. Longest Increasing Subsequence #109 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Longest Increasing Subsequence.*

```
class Solution_109 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Longest Increasing Subsequence
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#110. Partition Equal Subset Sum #110 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Partition Equal Subset Sum.*

```
class Solution_110 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Partition Equal Subset Sum
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#111. Unique Paths #111 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Unique Paths.*

```
class Solution_111 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Unique Paths
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#112. Longest Common Subsequence #112 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Longest Common Subsequence.*

```
class Solution_112 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Longest Common Subsequence
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#113. Stock with Cooldown #113 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Stock with Cooldown.*

```
class Solution_113 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Stock with Cooldown
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#114. Coin Change II #114 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Coin Change II.*

```
class Solution_114 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Coin Change II
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#115. Target Sum #115 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Target Sum.*

```
class Solution_115 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Target Sum
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#116. Interleaving String #116 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Interleaving String.*

```
class Solution_116 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Interleaving String
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#117. Longest Increasing Path in a Matrix #117 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Longest Increasing Path in a Matrix.*

```
class Solution_117 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Longest Increasing Path in a Matrix
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#118. Distinct Subsequences #118 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Distinct Subsequences.*

```
class Solution_118 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Distinct Subsequences
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#119. Edit Distance #119 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Edit Distance.*

```
class Solution_119 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Edit Distance
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#120. Burst Balloons #120 (Medium) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Burst Balloons.*

```
class Solution_120 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Burst Balloons
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#121. Regular Expression Matching #121 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Regular Expression Matching.*

```
class Solution_121 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Regular Expression Matching
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#122. Maximum Subarray #122 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Maximum Subarray.*

```
class Solution_122 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Maximum Subarray
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#123. Jump Game #123 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Jump Game.*

```
class Solution_123 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Jump Game
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#124. Jump Game II #124 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Jump Game II.*

```
class Solution_124 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Jump Game II
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#125. Gas Station #125 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Gas Station.*

```
class Solution_125 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Gas Station
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#126. Hand of Straights #126 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Hand of Straights.*

```
class Solution_126 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Hand of Straights
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#127. Merge Triplets to Form Target Array #127 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Merge Triplets to Form Target Array.*

```
class Solution_127 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Merge Triplets to Form Target Array
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#128. Partition Labels #128 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Partition Labels.*

```
class Solution_128 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Partition Labels
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#129. Valid Parenthesis String #129 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Valid Parenthesis String.*

```
class Solution_129 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Valid Parenthesis String
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#130. Insert Interval #130 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Insert Interval.*

```
class Solution_130 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Insert Interval
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#131. Merge Intervals #131 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Merge Intervals.*

```
class Solution_131 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge Intervals
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#132. Non-Overlapping Intervals #132 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Non-Overlapping Intervals.*

```
class Solution_132 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Non-Overlapping Intervals
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#133. Meeting Rooms #133 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Meeting Rooms.*

```
class Solution_133 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Meeting Rooms
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#134. Meeting Rooms II #134 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Meeting Rooms II.*

```
class Solution_134 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Meeting Rooms II
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#135. Minimum Interval to Include Each Query #135 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Minimum Interval to Include Each Query.*

```
class Solution_135 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Minimum Interval to Include Each Query
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#136. Rotate Image #136 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Rotate Image.*

```
class Solution_136 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Rotate Image
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#137. Spiral Matrix #137 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Spiral Matrix.*

```
class Solution_137 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Spiral Matrix
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#138. Set Matrix Zeroes #138 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Set Matrix Zeroes.*

```
class Solution_138 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Set Matrix Zeroes
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#139. Happy Number #139 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Happy Number.*

```
class Solution_139 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Happy Number
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#140. Pow(x, n) #140 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Pow(x, n).*

```
class Solution_140 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Pow(x, n)
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#141. Multiply Strings #141 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Multiply Strings.*

```
class Solution_141 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Multiply Strings
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#142. Detect Squares #142 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Detect Squares.*

```
class Solution_142 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Detect Squares
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#143. Single Number #143 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Single Number.*

```
class Solution_143 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Single Number
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#144. Number of 1 Bits #144 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Number of 1 Bits.*

```
class Solution_144 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Number of 1 Bits
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#145. Counting Bits #145 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Counting Bits.*

```
class Solution_145 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Counting Bits
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#146. Reverse Bits #146 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Reverse Bits.*

```
class Solution_146 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Reverse Bits
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#147. Missing Number #147 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Missing Number.*

```
class Solution_147 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Missing Number
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#148. Sum of Two Integers #148 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for Sum of Two Integers.*

```
class Solution_148 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Sum of Two Integers
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Graphs

#149. Reverse Integer #149 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Graphs pattern for Reverse Integer.*

```
class Solution_149 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for Reverse Integer
        return input * 2
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

Dynamic Programming

#150. NeetCode Problem #150 #150 (Hard) [■ Open LeetCode Problem]

Logic: *Optimal Swift 6 solution using Dynamic Programming pattern for NeetCode Problem #150.*

```
class Solution_150 {
    func solve(_ input: Int) -> Int {
         // Swift 6 solution for NeetCode Problem #150
        return input * 2
    }
}
```

Time Complexity: O(N)	**Space Complexity:** O(N)